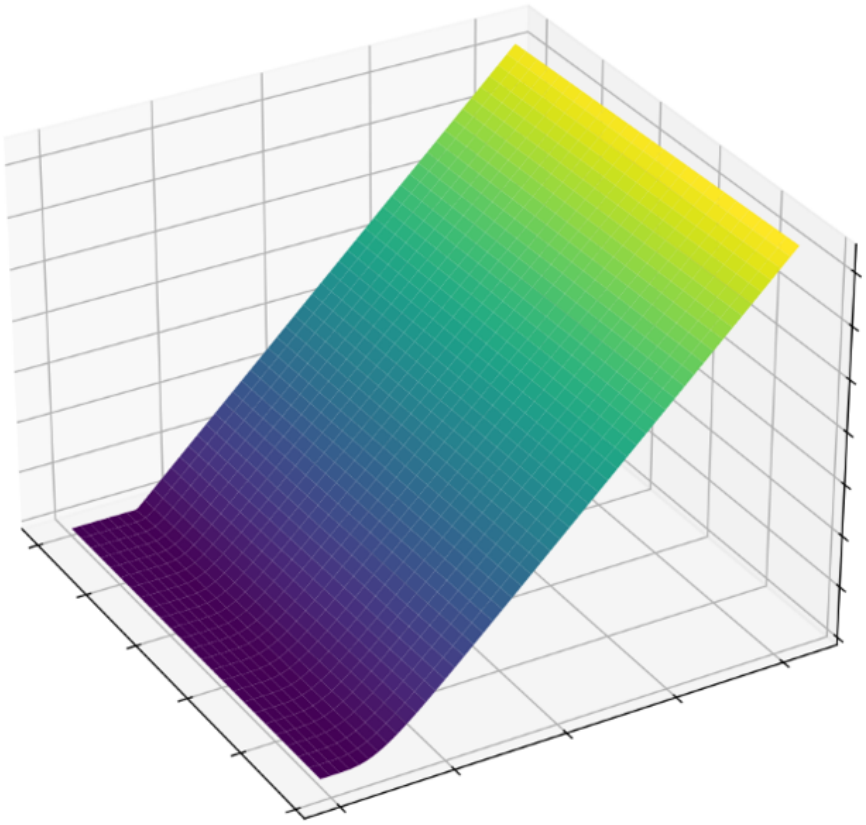
The Black-Scholes Equation with both Explicit and Implicit Solvers

Theo Gumbley



May 19, 2026

Contents

1 Introduction

This document covers the numerical methods and mathematical derivations used to find an approximate solution to the 1D Black-Scholes equation. These mathematical formulations are applied in the python code. This project serves as an opportunity for me to learn and apply knowledge, gained from my background in aerospace engineering and computational fluid dynamics, to a financial modeling problem.

The 1D Black-Scholes equation models the fair price of a European-style derivative that depends on a single underlying asset price and the time at which the valuation is taking place.

2 The Black-Scholes PDE

The 1D Black-Scholes PDE is defined below [1].

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0 \tag{1}$$

The terms of the equation are defined as follows. V, the option value, t, the time at which the option valuation takes place, σ, the annual volatility, S, the underlying asset price and r, the risk free rate.

The equation enforces no-arbitrage as the risk free rate requires a perfectly hedged portfolio (delta hedged). A delta hedged portfolio has no exposure to the underlying random fluctuations in asset value, therefore it grows at the risk free rate, r.

2.1 Key Equation Assumptions

- The short-term interest rate (risk free rate) is known and constant.
- The underlying asset price follows a random walk in continuous time.
- The volatility, σ, is constant.
- The option is "European" in the sense that it can only be exercised at expiry.
- The stock (underlying asset) pays no dividends [1]

2.2 Boundary Conditions

The boundary conditions for a European call and put are defined as follows.

European Call:

$$V(0,t) = 0 \qquad V(S,t) = S - Ke^{-r(T-t)} \ \ as \ \ S \to \infty$$

European Put:

$$V(0,t) = Ke^{-r(T-t)} \qquad V(S,t) = 0 \ \ as \ \ S \to \infty$$

The S-grid on which the call or put option value is evaluated on through time ranges from zero to S_{max}, which is often a 3-5 times multiple of the strike price.

For a European call, when the underlying asset price, S, reaches zero this represents a deep out of the money call option and can be approximated to be of zero value. As the asset price becomes large and tends to S_{max}, this represents a deep in the money call option. In this case the option is likely to be exercised and so behaves like a forward contract, where you have the obligation to buy or sell an asset at a predetermined price and time. Its value therefore becomes the asset price minus the strike price, discounted at the risk-free rate, over the remaining time to expiry.

For a European put, an underlying asset price of zero represents a deep in the money put option and so too behaves like a forward contract, as it likely will be exercised, with an option value of the strike price, discounted at the risk-free rate. An underlying asset price of S_{max} represents a deep out of the money put option and can be approximated to be zero.

2.3 Terminal Conditions

At expiry for a European call, if the strike price has not been reached by the underlying asset price, then the option has no value as there is no available time for price movement. Similarly, for a European put the option only has value if the underlying asset price falls below the strike price by expiry. Therefore, the following terminal conditions can be applied at expiry when $t = T$, T here being the time of expiry. The payoff functions above specify the option value at expiry t=T. In the forward in time formulation of the PDE these are referred to as terminal conditions. However, since the PDE is solved backwards in

time in this case, marching from t=T to t=0, they serve as the initial conditions for the numerical scheme.

European Call:

$$V = max(S - K, 0)$$

European Put:

$$V = max(K - S, 0)$$

3 Central Scheme Spatial Discretisation

3.1 Central Difference Approximation of the Derivatives

The terms $\frac{\partial V}{\partial S}$ and $\frac{\partial^2 V}{\partial S^2}$ of the Black-Scholes equation are approximated using a central difference approximation. This comes from a Taylor's expansion around the point S_i on the uniform S-grid with step size ΔS.

For a point to the right on the grid, $i + 1$, at time level, n, the Taylor's expansion is as follows.

$$V(S_{i+1}) = V(S_i) + \Delta S\frac{\partial V}{\partial S} + \frac{\Delta S^2}{2!}\frac{\partial^2 V}{\partial S^2} + \frac{\Delta S^3}{3!}\frac{\partial^3 V}{\partial S^3} + \frac{\Delta S^4}{4!}\frac{\partial^4 V}{\partial S^4} + \mathcal{O}(\Delta S^5)$$

For a point to the left on the grid, $i - 1$, the Taylor's expansion is as follows.

$$V(S_{i-1}) = V(S_i) - \Delta S\frac{\partial V}{\partial S} + \frac{\Delta S^2}{2!}\frac{\partial^2 V}{\partial S^2} - \frac{\Delta S^3}{3!}\frac{\partial^3 V}{\partial S^3} + \frac{\Delta S^4}{4!}\frac{\partial^4 V}{\partial S^4} + \mathcal{O}(\Delta S^5)$$

To derive the term $\frac{\partial V}{\partial S}$, subtract the left expansion from the right, and remove higher order terms.

$$V(S_{i+1}) - V(S_{i-1}) = 2\Delta S\frac{\partial V}{\partial S} + 2\frac{\Delta S^3}{3!}\frac{\partial^3 V}{\partial S^3} + \mathcal{O}(\Delta S^3)$$

This is then rearranged to the following.

$$\frac{\partial V}{\partial S} = \frac{V(S_{i+1}) - V(S_{i-1})}{2\Delta S} \tag{2}$$

Which has a truncation error of $\mathcal{O}(\Delta S^2)$.

To derive the term $\frac{\partial^2 V}{\partial S^2}$, add the left expansion to the right, and remove higher order terms.

$$V(S_{i+1}) + V(S_{i-1}) = 2V(S_i) + 2\frac{\Delta S^2}{2!}\frac{\partial^2 V}{\partial S^2} + \mathcal{O}(\Delta S^4)$$

This is then rearranged to the following.

$$\frac{\partial^2 V}{\partial S^2} = \frac{V(S_{i+1}) + V(S_{i-1}) - 2V(S_i)}{\Delta S^2} \tag{3}$$

Which has a truncation error of $\mathcal{O}(\Delta S^2)$.

3.2 Applying the Central Difference to Black-Scholes

Substituting equations 2 and 3 into the Black-Scholes equation and rearranging for the $\frac{\partial V}{\partial t}$ term, we arrive at the following.

$$\frac{\partial V}{\partial t} = rV(S_i) - rS_i\frac{V(S_{i+1}) - V(S_{i-1})}{2\Delta S} - \frac{1}{2}\sigma^2 S_i^2\frac{V(S_{i+1}) + V(S_{i-1}) - 2V(S_i)}{\Delta S^2}$$

Then simplifying by defining the below coefficients, we finally reach the following equation.

$$a = \frac{1}{2}\sigma^2\frac{S_i^2}{\Delta S^2} - r\frac{S_i}{2\Delta S}$$

$$b = -\sigma^2\frac{S_i^2}{\Delta S^2} - r$$

$$c = \frac{1}{2}\sigma^2\frac{S_i^2}{\Delta S^2} + r\frac{S_i}{2\Delta S}$$

$$\frac{\partial V}{\partial t} = -(aV(S_{i-1}) + bV(S_i) + cV(S_{i+1})) \tag{4}$$

This equation can then be used by a time integration scheme, such as Runge-Kutta (RK4) or Crank-Nicholson, to solve for the option value at each time-step over the period until expiry.

4 Time Integration Schemes

4.1 Runge-Kutta (RK4)

The RK4 solver was my first port of call for solving Black-Scholes, due to its simplicity to implement and accuracy.

The RK4 scheme comes from the method of lines. The derivative $\frac{\partial V}{\partial t}$ as in equation (4), is equivalent to **f(V)**, where V is a vector containing the option values, V, for every interior node on the 1D S-grid. The integral of the update equation below is approximated by evaluating **f(V)** at 4 locations between t and $t + \Delta t$. This gives the solver 4^{th} order accuracy in time [2].

$$\frac{\partial V}{\partial t} = \mathbf{f(V)}$$

$$\mathbf{V}(t + \Delta t) = \mathbf{V}(t) + \int_{t}^{t+\Delta t} \mathbf{f}(\mathbf{V}(s))ds$$

the slopes calculated at each of the 4 locations are combined using Simpson's rule weights to approximate the full integral.

The final update equation for the Black-Scholes equation, as is used in the python code is the following [2].

$$\mathbf{V}^{n+1} = \mathbf{V}^{n} + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{5}$$

Where the coefficients (slopes of the 4 locations) are calculated in order as follows.

$$k_1 = \mathbf{f}(\mathbf{V})$$

$$k_2 = \mathbf{f}(\mathbf{V} + \frac{k_1}{2})$$

$$k_3 = \mathbf{f}(\mathbf{V} + \frac{k_2}{2})$$

$$k_4 = \mathbf{f}(\mathbf{V} + k_3)$$

Since the RHS of equation (4) has no explicit time dependence, **f(V)** is written rather than **f(t,V)** as would be the case for a general ODE system.

An RK4 solver is explicit and therefore is governed by the Courant–Friedrichs–Lewy

(CFL) condition [4]. The CFL condition Black-Scholes and an RK4 solver is derived using Von Neumann Analysis [3], which provides the formula below.

$$\Delta t \leq \frac{\Delta S^2}{\sigma^2 S_{max}^2}$$

This stability formula arrives due to the dominant diffusive term, $\frac{\partial^2 V}{\partial S^2}$, of the Black-Scholes equation, and by satisfying this constraint it automatically satisfies the stability constraint of the RK4 solver. This is true for other explicit schemes, but is not a given and should be checked. If this timestep limit is exceeded, the residuals will diverge due to the numerical instability of the solution.

4.2 Crank-Nicholson (CN)

The Crank-Nicholson scheme was next considered as the scheme is implicit and not bounded by the timestep stability constraint of explicit solvers like RK4. The CN formula is detailed below [5].

$$\frac{V_i^{n+1} - V_i^n}{\Delta t} = \frac{1}{2}L[V_i^{n+1}] + \frac{1}{2}L[V_i^n]$$

L is the spatial operator here and looks like equation (4) from before with the same coefficient definitions, as seen below.

$$L[V_i^n] = -(aV_{i-1}^n + bV_i^n + cV_{i+1}^n)$$

Substituting the above equation into the CN equation provides the following.

$$\frac{V_i^{n+1} - V_i^n}{\Delta t} = -\frac{1}{2}(aV_{i-1}^{n+1} + bV_i^{n+1} + cV_{i+1}^{n+1}) - \frac{1}{2}(aV_{i-1}^n + bV_i^n + cV_{i+1}^n)$$

Then by multiplying both sides of the equation by Δt and moving all $n+1$ terms to the left and n terms to the right, we reach the following.

$$-\frac{a\Delta t}{2}V_{i-1}^{n+1} + V_i^{n+1}(1 - \frac{b\Delta t}{2}) + \frac{c\Delta t}{2}V_{i+1}^{n+1} = \frac{a\Delta t}{2}V_{i-1}^n + V_i^n(1 + \frac{b\Delta t}{2}) + \frac{c\Delta t}{2}V_{i+1}^n$$

We can then assign α, β, and γ coefficients to the n+1 side of the equation and $\tilde{\alpha}$, $\tilde{\beta}$, and $\tilde{\gamma}$ coefficients to the n side of the equation. This gives the following

final equation [5].

$$\alpha_i V_{i-1}^{n+1} + \beta_i V_i^{n+1} + \gamma_i V_{i+1}^{n+1} = \tilde{\alpha}_i V_{i-1}^{n} + \tilde{\beta}_i V_i^{n} + \tilde{\gamma}_i V_{i+1}^{n} \tag{6}$$

The coefficients for each side of the equation are formally defined below.

LHS Coefficients	**RHS Coefficients**
$\alpha_i = -a_i \frac{\Delta t}{2}$	$\tilde{\alpha}_i = -\alpha_i$
$\beta_i = 1 - \frac{\Delta t}{2} b_i$	$\tilde{\beta}_i = 2 - \beta_i$
$\gamma_i = -c_i \frac{\Delta t}{2}$	$\tilde{\gamma}_i = -\gamma_i$

Applying equation (6) to every option value at every interior node in the S-grid, a matrix system of equations is created with the form seen below.

$$A \cdot V_{interior}^{n+1} = B \cdot V_{interior}^{n} + d^n$$

Here, A and B are $N-2 \times N-2$ matrices with the structure seen below, and d^n is the boundary correction vector for time level, n.

$$A = \begin{bmatrix} \beta_1 & \gamma_1 & 0 & \cdots & 0 \\ \alpha_2 & \beta_2 & \gamma_2 & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \alpha_{N-3} & \beta_{N-3} & \gamma_{N-3} \\ 0 & \cdots & 0 & \alpha_{N-2} & \beta_{N-2} \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} \tilde{\beta}_1 & \tilde{\gamma}_1 & 0 & \cdots & 0 \\ \tilde{\alpha}_2 & \tilde{\beta}_2 & \tilde{\gamma}_2 & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \tilde{\alpha}_{N-3} & \tilde{\beta}_{N-3} & \tilde{\gamma}_{N-3} \\ 0 & \cdots & 0 & \tilde{\alpha}_{N-2} & \tilde{\beta}_{N-2} \end{bmatrix}$$

The boundary correction vector accounts for the effect of the boundary condition nodes at either end of the S-grid on the matrix system. For a European call, the boundary conditions at either end of the S-grid are given by $V_0^n = 0$ and $V_{N-1}^n = S_{max} - Ke^{-r(T-t_n)}$. Only the first and last interior nodes are affected, as they have neighbours at the boundary.

The correction vector is therefore a column vector of length $N-2$ of the following form.

$$d^n = \begin{bmatrix} \tilde{\alpha}_1 V_0^n - \alpha_1 V_0^{n+1} \\ 0 \\ \vdots \\ 0 \\ \tilde{\gamma}_{N-2} V_{N-1}^n - \gamma_{N-2} V_{N-1}^{n+1} \end{bmatrix}$$

The upper and lower entries take into account the alpha and gamma terms, which are not represented in the A and B matrices defined above at both time levels n and $n+1$.

This system of equations can then be solved using a Thomas algorithm. However, in the python code the Scipy "solve banded" approach is taken, a python optimised solver appropriate for the tri-diagonal system of equations formed here.

5 Validation and Results

5.1 Closed-Form Black-Scholes Equations

The closed-form Black-Scholes equations were used to validate solutions from the PDE solvers. The closed form equations arise from a change of variable from S, the underlying asset price to $\log(\frac{S}{K})$, and scaling the time, t. There is a known analytical solution for the heat equation, involving a convolution integral with the Gaussian kernel. This method once transformed back into the original variables provides the following analytical solutions for a European call and put.

European Call:

$$V(S,t) = SN(d_1) - Ke^{-r(T-t)}N(d_2)$$

$$\text{where } d_1 = \frac{\ln(\frac{S}{K}) + (r + \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}} \text{ and } d_2 = d_1 - \sigma\sqrt{T-t}$$

European Put:

$V(S,t) = Ke^{-r(T-t)}N(-d_2) - SN(-d_1)$ where d_1 and d_2 are defined as before.

Here, $N(x)$ represents the cumulative normal distribution function.

These equations are valid for the assumptions of the Black-Scholes equations stated previously, and are both faster and exact when finding solutions for European options. However, the closed-form equations are not valid for American options, where the options contract is exercisable at any time before expiry. This creates a floating boundary condition and makes a PDE solver necessary. Other examples including Asian options similarly cause the closed-form equations to break down.

5.2 Results

Both time integration schemes were tested on both a European Put and Call and compared with the closed-form analytical solution. The plots below picture solutions for a European call option value, over a range of underlying asset prices, with a period of 5 years left until expiry.

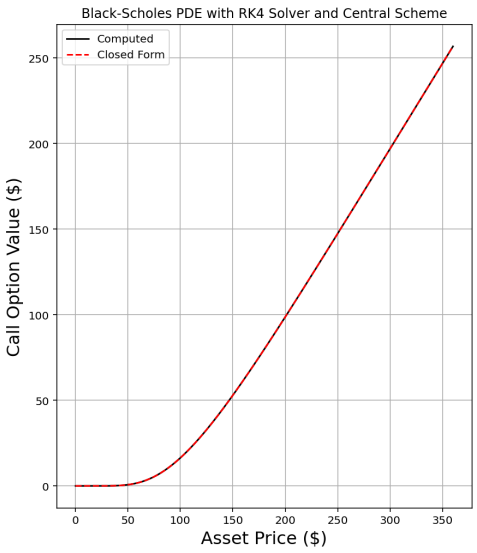


Figure 1: RK4 Solver European Call Solution

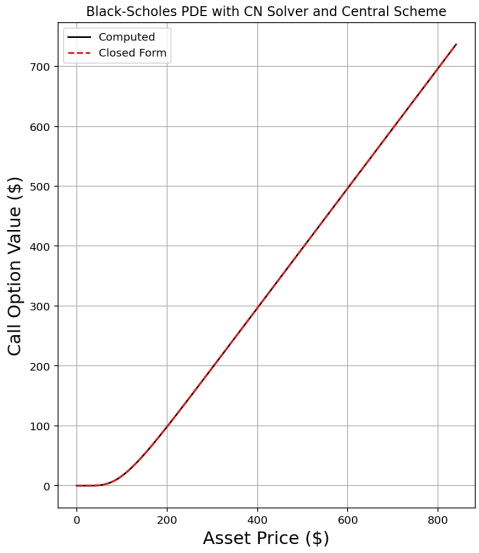


Figure 2: CN Solver European Call Solution

The convergence of the residuals of each solver is plotted below in figures 3 and 4.

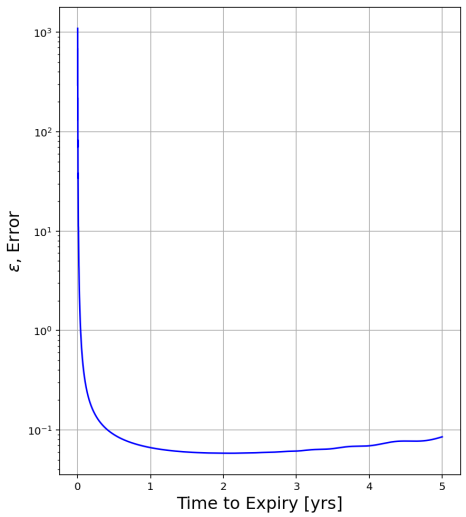


Figure 3: RK4 Solver European Call Error Plot

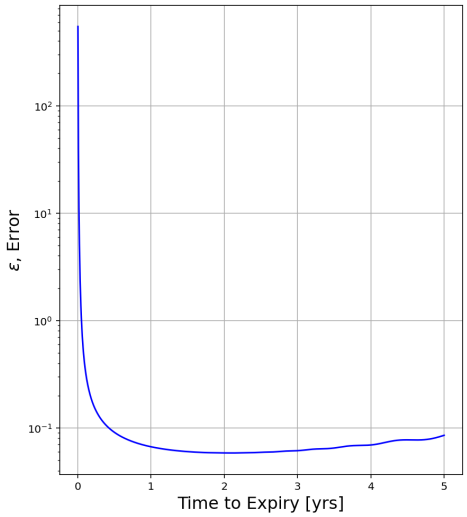


Figure 4: CN Solver European Call Error Plot

The solution for both solvers sits on top of the closed-form results, suggesting a correct implementation of the numerical schemes. The error of the solution to the closed form equation, plotted in figures 3 and 4 above, was calculated using the following equation.

$$\epsilon = \frac{|V_i^{numerical} - V_i^{closedform}|}{|V_i^{closedform}| + \delta} \tag{7}$$

Here, δ is a very small value added to prevent division by zero. A table of input parameters is given below.

Parameter	**RK4 Value**	**CN Value**
Annualised volatility, σ	0.2	0.2
Risk Free Rate, r	0.03	0.03
Time to Expiry (years), T	5	5
Strike Price ($), K	120	120
S-grid Points, n	200	200
Required Time Steps, m	8805	1500

Table 1: Table of Input Parameters

5.3 Computing Greeks

The "Greeks" Δ, Γ, and Θ were computed from the results obtained above. The definitions of the Greeks are provided below.

$$\Delta = \frac{\partial V}{\partial S}$$

$$\Gamma = \frac{\partial^2 V}{\partial S^2} = \frac{\partial \Delta}{\partial S}$$

$$\Theta = \frac{\partial V}{\partial t}$$

Δ and Γ allow you to understand the rate at which the option value is changing and whether the rate of change of the option price is increasing or decreasing. Θ, allows you to understand, for a particular underlying asset price, how the option value will change over time, as you approach the expiration date. These quantifications give insight into the vulnerability to change of the option price you have predicted at the current time but also in the future. In the case of the python code, Δ and Γ were computed using the "numpy.gradient" function which uses a 2^{nd} order central difference approximation [6]. To calculate Θ, The option value for the given underlying asset price was interpolated at each time step and appended to an array. The "numpy.gradient" function was then used to compute an approximation for the derivative.

5.4 Time Integration Scheme Comparison

Whilst the relative error plotted in figures 3 and 4 above shows a very similar result the computational of cost between the explicit RK4 and implicit CN schemes is vastly different.

Scheme	Wall Clock Time (s)
Runge-Kutta (RK4)	36.14
Crank-Nicholson (CN)	0.25

Table 2: Comparison of Computational Cost

The difference in computational cost comes from the largely reduced number of time steps required by an implicit scheme like CN to remain stable when compared to an explicit RK4 scheme. This is a well known result in the world of quantitative finance. It should be noted that the CN solver remains stable

for a lower number of time steps than stated in table 1, but suffers in the accuracy of the solution. The difference in computational cost would be expected to be amplified by higher dimensions, a non-uniform grid, and other additional parameters, all of which introduce stiffness into the problem which prohibits the time step of an explicit solver.

References

[1] Fischer Black and Myron Scholes. “The Pricing of Options and Corporate Liabilities”. In: *Journal of Political Economy* 81 (1973), pp. 637–654.

[2] John Charles Butcher. *Numerical Methods for Ordinary Differential Equations*. 2nd. Chichester: John Wiley & Sons, 2008. ISBN: 978-0-470-72335-7.

[3] Jule G. Charney, Ragnar Fjørtoft, and John von Neumann. “Numerical Integration of the Barotropic Vorticity Equation”. In: *Tellus* 2 (1950), pp. 237–254.

[4] Richard Courant, Kurt Friedrichs, and Hans Lewy. “On the Partial Difference Equations of Mathematical Physics”. In: *IBM Journal of Research and Development* 11.2 (1967). English translation of the original 1928 German article in *Mathematische Annalen*, vol. 100, pp. 32–74, pp. 215–234.

[5] John Crank and Phyllis Nicolson. “A Practical Method for Numerical Evaluation of Solutions of Partial Differential Equations of the Heat-Conduction Type”. In: *Mathematical Proceedings of the Cambridge Philosophical Society* 43 (1947), pp. 50–67.

[6] NumPy. “"numpy.gradient" - NumPy API Reference”. In: (). URL: https://numpy.org/doc/stable/reference/generated/numpy.gradient.html.